
Contents

1 Introduction	2
2 Prétraitement des images	1
2.1 Éclaircissement de l'image	1
2.2 Conversion en niveaux de gris	1
2.3 Binarisation	2
2.4 Chargement d'images dans des fichiers	2
3 Extraction du texte avec OCR	3
3.1 OCR (Reconnaissance Optique de Caractères)	3
3.2 Étapes de l'application de l'OCR	3
3.2.1 Définition des dossiers d'entrée et de sortie	3
3.2.2 Découpage des documents par zones	4
3.2.3 Application de l'OCR avec pytesseract	4
4 Extraction des données à l'aide des expressions régulières	5
4.1 Invoice Number	6
4.2 Date	6
4.3 Net Worth,VAT And Gross Worth	6
4.4 Tax ID	6
4.5 IBAN	7
4.6 State city zip code	7
4.7 Seller Name	7
4.8 Client name	7
5 Extraction des données structurées	7
5.1 Etapes suivies pour la structuration de données:	8
5.1.1 Définition de la fonction "extract_data"	8
5.1.2 Définition de la fonction process_folder	8
6 Structuration des résultats	10
6.1 Fichier Csv	10
6.2 Fichier Json	11
6.3 Base de données	11
6.3.1 Requêtes SQL sur la base	12
7 Interface utilisateur	13
7.1 Fonctionnalités Principales	14
7.2 Démonstration Vidéo	14
8 Conclusion	15

1 Introduction

Dans un contexte professionnel où la numérisation et l'automatisation des processus deviennent incontournables, de nombreuses entreprises continuent de recevoir un grand nombre de factures au format papier ou scanné. Le traitement manuel de ces documents, notamment l'extraction d'informations essentielles telles que le numéro de facture, la date, le montant ou la TVA, représente une tâche chronophage et sujette aux erreurs humaines.

Ce projet vise à répondre à cette problématique en automatisant l'extraction d'informations clés à partir de documents scannés, en particulier les factures. Pour cela, nous faisons appel à des techniques de *Reconnaissance Optique de Caractères* (OCR), qui permettent de convertir le texte contenu dans des images ou des fichiers PDF en texte numérique exploitable. Toutefois, le texte obtenu via (OCR) est souvent brut et non structuré ; il est donc nécessaire de mettre en place un traitement complémentaire pour extraire de manière fiable les données recherchées.

L'objectif principal de ce projet est donc de développer une solution capable de traiter automatiquement des documents numérisés, d'en extraire les données pertinentes à l'aide d'expressions régulières, puis de structurer ces données dans un format facilement exploitable tel que le CSV ou le JSON. Pour ce faire, différentes étapes seront abordées, allant du prétraitement des images à l'implémentation de l'(OCR), en passant par l'analyse du texte extrait et le développement éventuel d'une interface utilisateur.

2 Prétraitement des images

Le **prétraitement** peut inclure différentes opérations comme la **conversion en niveaux de gris**, la **binarisation**, le redimensionnement etc. Ces transformations visent à améliorer la lisibilité du texte présent dans l'image afin d'optimiser son interprétation par l'**OCR** et améliorer ses performances.

Pour cela, nous utilisons le module `Image` de la bibliothèque **Pillow**, qui permet de manipuler les fichiers image facilement.

Voici l'importation de la bibliothèque :

Librairie Pillow

```
from PIL import Image
```

2.1 Éclaircissement de l'image

Pour améliorer la qualité de la reconnaissance (**OCR**), il est parfois nécessaire d'ajuster la luminosité de l'image. Cette opération permet d'augmenter le contraste entre le texte (souvent noir) et le fond (souvent blanc), ce qui facilite la détection des caractères. En éclaircissant légèrement l'image, on rend les zones sombres plus marquées tout en atténuant les variations de fond.

L'ajustement a été réalisé à l'aide de la classe **ImageEnhance.Brightness** de la bibliothèque `Pillow`, comme montré dans le code ci-dessous :

Brightening

```
brightener=ImageEnhance.Brightness(img)
img=brightener.enhance(1.5)
```

SUMMARY				
	VAT [%]	Net worth	VAT	Gross worth
	10%	28,24	2,82	31,06
Total		\$ 28,24	\$ 2,82	\$ 31,06

(a) Avant éclaircissement

SUMMARY				
	VAT [%]	Net worth	VAT	Gross worth
	10%	28,24	2,82	31,06
Total		\$ 28,24	\$ 2,82	\$ 31,06

(b) Après éclaircissement

Figure 1: Comparaison des images avant et après éclaircissement

Note :

Comme on peut le constater, un simple éclaircissement de l'image suffit à la rendre plus lisible. Il n'est donc pas nécessaire d'appliquer de nombreux effets supplémentaires, car cela risquerait de dégrader la qualité de l'image à cause d'un surtraitement, entraînant ainsi une perte de pixels importants pour l'extraction d'informations.

2.2 Conversion en niveaux de gris

Les images en couleur sont généralement en **RGB**, avec trois composantes (rouge, vert, bleu) allant de 0 à 255. Cette diversité de couleurs peut compliquer le traitement, notamment lors de la **binarisation**, car chaque nuance pourrait être interprétée différemment.

Afin de simplifier l'analyse et de préparer efficacement l'image à l'étape suivante, nous avons converti les images en niveaux de gris. Cette transformation permet de représenter chaque pixel par une seule valeur d'intensité lumineuse (de 0 à 255), facilitant ainsi la distinction entre le fond et le texte.

La conversion en niveaux de gris a été réalisée à l'aide de la méthode suivante :

```
gray = img.convert("L")
```

Cette opération est essentielle pour normaliser l'image et réduire la variabilité des informations visuelles inutiles avant la binarisation.

2.3 Binarisation

Nous avons effectué une **binarisation** de l'image, ce qui signifie qu'elle ne contient désormais plus que deux couleurs possibles : 0 pour le noir et 255 pour le blanc. Cette opération repose sur l'application d'un seuil fixe, ici fixé à 128. Concrètement, chaque pixel dont la valeur de niveau de gris est strictement supérieure à 128 est converti en blanc (255), tandis que les autres sont convertis en noir (0).

```
def seuil_pixel(p):  
    if p > 128:  
        return 255 # Blanc  
    else:  
        return 0   # Noir  
  
binaire = gris.point(seuil_pixel)
```

2.4 Chargement d'images dans des fichiers

Nous avons regroupé l'ensemble des étapes de traitement précédemment décrites (**éclaircissement, passage en niveaux de gris, binarisation**) dans une fonction appelée **process_image**. Cette fonction prend en entrée deux arguments : le chemin de l'image à traiter et le chemin où enregistrer l'image finale. Elle permet ainsi d'automatiser le traitement d'un grand nombre d'images de manière structurée et réutilisable.

On peut montrer ça par le code python qui suit :

image processing function

```
def process_image(image_path, save_path):  
    img = Image.open(image_path)  
  
    # Brightening  
    brightener = ImageEnhance.Brightness(img)  
    img = brightener.enhance(1.5)  
  
    # Grayscale  
    gray = img.convert("L")  
  
    # Binarization  
    def threshold_pixel(p):  
        return 255 if p > 128 else 0  
  
    binary = gray.point(threshold_pixel)  
  
    # Save result  
    binary.save(save_path)
```

Par la suite, nous avons utilisé la bibliothèque **OS** afin de faciliter l'importation et l'exportation des images entre deux dossiers : un dossier source (*batch_1*) contenant les images originales, et un dossier de destination (*Processed_images*) pour sauvegarder les résultats.

```
input_folder = "batch_1"
output_folder = "Processed_images"
```

3 Extraction du texte avec OCR

3.1 OCR (Reconnaissance Optique de Caractères)

L'OCR est une technologie permettant de convertir du texte imprimé ou manuscrit, présent dans des images ou des documents scannés, en texte numérique exploitable. Cette technologie analyse les caractères visuels, identifie les formes et les compare à des modèles de caractères préalablement enregistrés pour extraire le contenu textuel de manière automatique.

3.2 Étapes de l'application de l'OCR

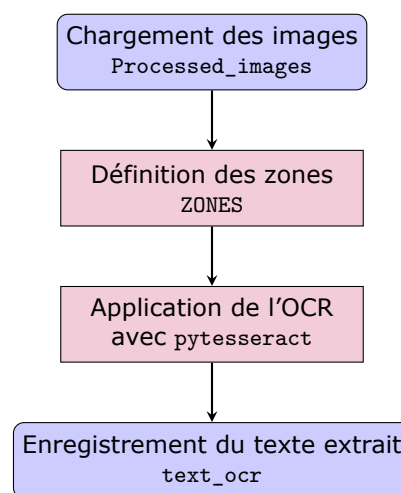


Figure 2: Schéma des étapes principales de l'application de l'OCR

Les principales étapes de l'application de la reconnaissance optique de caractères (OCR) sont les suivantes :

3.2.1 Définition des dossiers d'entrée et de sortie

Avant d'appliquer le traitement OCR, à l'aide de la bibliothèque **OS** qui permet de gérer les chemins de fichiers, et les déplacer de créer des répertoires pour l'importation et l'exportation des documents traités, On commence par définir les chemins vers deux dossiers principaux :

- **Dossier d'entrée** (`Processed_images`) : ce dossier contient l'ensemble des images qui ont été préalablement prétraitées et qui sont prêtes pour l'extraction de texte.
- **Dossier de sortie** (`text_ocr`) : ce dossier est destiné à accueillir les fichiers texte générés après l'application de l'OCR. Chaque fichier contient le texte extrait d'une image, structuré selon les différentes zones du document.

Cette organisation permet de bien séparer les données brutes des résultats obtenus, ce qui facilite le suivi du traitement, la vérification des résultats et la réutilisation des fichiers.

Note: Si l'un des fichiers est manquant, une erreur sera levée, il faut vérifier l'existence de ces fichiers afin d'éviter toute interruption non contrôlée lors de l'exécution.

3.2.2 Découpage des documents par zones

Chaque image est divisée en quatre zones prédéfinies (*header*, *seller_info*, *client_info*, *body_content*) selon des coordonnées spécifiques, afin de localiser et isoler les parties importantes d'un document (comme les titres, montants, dates, etc.). Ces coordonnées ont été déterminées à l'aide de Photoshop. Cela permet d'améliorer la précision de l'extraction de texte en se concentrant uniquement sur les zones pertinentes, évitant ainsi les erreurs dues à du contenu inutile ou du bruit visuel. Cette approche est possible uniquement parce que nos documents sont stables et que les informations se trouvent toujours aux mêmes emplacements. Si les documents n'étaient pas aussi homogènes, nous n'aurions pas pu adopter cette méthode.

```
ZONES = {  
  "header": (0, 20, 1600, 280),  
  "seller_info": (0, 281, 674, 750),  
  "client_info": (674, 281, 1600, 750),  
  "body_content": (0, 750, 1600, 2290)  
}
```

The diagram shows an invoice with four distinct zones outlined by dashed lines and color-coded in a legend at the bottom:

- Header Zone (Pink):** Contains 'Invoice no: 17045625' and 'Date of issue: 11/01/2017'.
- Seller Zone (Green):** Contains 'Seller: Bass-Petersen, 88013 Keith Orchard, Port Jenniferfurt, ID 01419' and tax information.
- Client Zone (Yellow):** Contains 'Client: Davidson-Martinez, 36262 Walters Vista, Evansstad, ND 36644' and tax information.
- Body Zone (Blue):** Contains an 'ITEMS' table and a 'SUMMARY' table.

ITEMS Table:

No.	Description	Qty	UM	Net price	Net worth	VAT [%]	Gross worth
1.	Cell: A Novel by Stephen King	4,00	each	4,49	17,96	10%	19,76
2.	A History of the Indians of the United States (The	1,00	each	4,49	4,49	10%	4,94
3.	Quilts of Illusion	1,00	each	5,79	5,79	10%	6,37

SUMMARY Table:

	VAT [%]	Net worth	VAT	Gross worth
	10%	28,24	2,82	31,06
Total		\$ 28,24	\$ 2,82	\$ 31,06

Legend:

- Header Zone
- Seller Zone
- Client Zone
- Body Zone

Figure 3: Répartition des zones a traité par l'OCR

3.2.3 Application de l'OCR avec pytesseract

Nous appliquons l'OCR à l'aide de la bibliothèque *pytesseract* sur l'ensemble des images contenues dans le dossier d'entrée. Chaque image est traitée individuellement afin d'en extraire le

texte qu'elle contient. Une fois l'extraction terminée, le texte obtenu est exporté et sauvegardé dans des fichiers distincts au sein du dossier de sortie, facilitant ainsi l'organisation et l'analyse des résultats.

Execution de l'OCR

```
text = pytesseract.image_to_string(zone_img, config=config)
```

Voici un exemple du text extrait grace a l'OCR

Invoice no: 17045625	3. Quilts of lilusion
Date of issue:	SUMMARY
Seller:	Total
Bass-Petersen	Qty
88013 Keith Orchard	4,00
Port Jenniferfurt, ID 01419	1,00
Tax Id: 903-94-7610	1,00
IBAN: GB43AYYU13392404742725	VAT [%]
ITEMS	10%
No. Description	11/01/2017
L. Cell: A Novel by Stephen King	UM
2. A History of the Indians of the United States (The	

Figure 4: Exemple du text extrait grace a l'OCR

4 Extraction des données à l'aide des expressions régulières

Dans cette section, nous avons utilisé les expressions régulières grâce à la bibliothèque **re** de Python. Les expressions régulières (ou *regex*) permettent de rechercher des motifs spécifiques dans du texte brut, ce qui est particulièrement utile pour extraire des informations structurées à partir du texte issu de l'OCR (comme des dates, des numéros, des adresses, etc.).

Nous nous sommes principalement appuyés sur deux fonctions de cette bibliothèque :

- **re.search** : cette fonction permet de trouver la **première occurrence** d'un motif dans une chaîne de caractères. Elle est utilisée lorsque l'on sait qu'il ne devrait y avoir qu'un seul élément correspondant, par exemple une date ou un numéro de facture.
- **re.findall** : cette fonction retourne **toutes les correspondances** d'un motif donné dans le texte. Elle est particulièrement utile pour capturer plusieurs éléments similaires, comme les montants financiers ou les identifiants fiscaux.

4.1 Invoice Number

Le numéro de facture est facilement identifiable : il est composé de 8 chiffres, suivis d'un saut de ligne. Étant donné qu'il n'y a qu'un seul numéro de facture par document, l'utilisation de la fonction `search` (et non `findall`) est suffisante pour extraire cette information.

```
regex=r'(\d{8})\n'

iban=re.search(regex,raw_text)
print(iban.group(1))
```

4.2 Date

La date est également unique dans le document, ce qui justifie l'utilisation de la fonction `search`. Son format est reconnaissable : deux chiffres pour le jour, suivis d'un /, puis deux chiffres pour le mois, et enfin quatre chiffres pour l'année (par exemple : 12/05/2024).

```
regex=r'\d{1,2}/\d{2}/\d{4}'

iban=re.search(regex,raw_text)
print(iban.group(0))
```

4.3 Net Worth,VAT And Gross Worth

Dans cette partie, seule la valeur du **Net Worth** (valeur nette) est extraite directement depuis le texte. Les autres valeurs financières sont dérivées à partir de celle-ci :

- La **TVA** est calculée comme 10% de la valeur nette :

$$\text{TVA} = \text{Net Worth} \times 0.1$$

- La **Gross Worth** (valeur brute) correspond à la somme de la valeur nette et de la TVA

$$\text{Gross Worth} = \text{Net Worth} + \text{TVA}$$

Puisque la première valeur apparaît avec \$ donc on peut utiliser cette expression :

```
regex = r'\nTotal \$s*([\s*\d*]+\,\d{2})'

name = re.findall(regex, raw_text)
name
float(name[0].strip().replace(",","."))
```

4.4 Tax ID

Une expression régulière est utilisée pour extraire les identifiants fiscaux au format standard (ex. : 976-94-0858). La fonction `findall()` récupère toutes les occurrences, et les deux premières sont stockées dans deux variables distinctes. Ce traitement suppose la présence exacte de deux identifiants dans le texte.

```
regex=r'Tax Id:\s*(\d+-\d{2,3}-\d+)'

tax_ids = re.findall(regex, raw_text)

# Unpack into two variables (works only if exactly 2 matches)
tax_id_1, tax_id_2 = tax_ids
print("First Tax ID:", tax_id_1)
print("Second Tax ID:", tax_id_2)
```

4.5 IBAN

Étant donné qu'un seul IBAN est présent par document, la fonction `search` est utilisée. Une fois trouvé, le résultat est affiché via `iban.group(1)`, qui renvoie uniquement la valeur de l'IBAN sans le texte précédent.

```
regex=r'IBAN:\s*(\w+)'

iban=re.search(regex,raw_text)
print(iban.group(1))
```

4.6 State city zip code

Cette expression régulière est utilisée pour extraire les informations de localisation, présentées sous la forme : Nom de la ville, Code de l'État (2 lettres), Code postal (5 chiffres). pour le client et le seller.

```
regex=r'\n([a-zA-Z\s]+),?\s*([A-Z]{2})\s*(\d{5})\n'

states=re.findall(regex,raw_text)
print(states)
# first one the city the second is the state and the last one is the zip code
```

Cette expression régulière permet d'extraire le nom et l'adresse du client à partir d'un format structuré. Elle recherche une section commençant par "Client:", suivie du nom (lettres, espaces, tirets) et de l'adresse (caractères alphanumériques et ponctuation). Les deux éléments extraits sont ensuite récupérés avec `re.findall()`.

4.7 Seller Name

```
regex = r'Seller:\n+([A-Za-z\s,-])\n+([A-Za-z0-9\s,.])+'

# the first one is hte name of the seller the second is the address of the seller
name = re.findall(regex, raw_text)
```

Cette expression régulière identifie le bloc "Seller:" suivi du nom du vendeur, puis de son adresse sur la ligne suivante. Le motif prend en compte les lettres, espaces, virgules et autres signes usuels. Les informations sont extraites avec `re.findall()` sous forme de couple (nom, adresse).

4.8 Client name

```
regex = r'Client:\n+([A-Za-z\s,-])\n+([A-Za-z0-9\s,.])+'

#the first one is hte name of the Clieen the second is the address of the Client

name = re.findall(regex, raw_text)
```

5 Extraction des données structurées

Nos avons enregistré Les données extraites dans un fichier **CSV** afin de les structurer et de faciliter leur analyse. Ce format tabulaire permet de regrouper toutes les informations clés (montant, date, coordonnées, etc.) pour chaque facture traitée de manière claire et exploitable

5.1 Etapes suivis pour la structuration de données:

5.1.1 Definition de la fonction "extract_data"

La fonction `extract_data` est conçue pour extraire des informations **structurées** à partir d'un texte brut (entré de la fonction `raw_text`) obtenu, par exemple, via un OCR. Elle utilise pour cela des expressions régulières pré-définies.

Voici un exemple sur une seule colonne "invoice number", on a fait de meme pour les autres colonnes (TAX_D, Seller_name....etc)

```
def extract_data(raw_text):  
    """Extract all data from raw text using the regex patterns"""  
    data = {}  
  
    # Invoice number  
    match = re.search(regex_patterns['invoice_number'], raw_text)  
    data['invoice_number'] = match.group(1) if match else None
```

```
# Invoice number  
match = re.search(regex_patterns['invoice_number'], raw_text)
```

Cette ligne de code permet de rechercher dans le texte une correspondance avec le motif associé au numéro de facture. Si une correspondance est trouvée, l'objet `match` contient les informations extraites, et la ligne suivante :

```
data['invoice_number'] = match.group(1) if match else None
```

permet de stocker la valeur extraite dans le dictionnaire `data`, ou bien `None` si aucun numéro de facture n'a été détecté. L'utilisation de `group(1)` indique qu'on extrait le premier groupe capturé par l'expression régulière, correspondant à la partie utile du motif (par exemple, un identifiant de facture comme "INV-2024-005").

5.1.2 Définition de la fonction process_folder

```
def process_folder(folder_path):  
    """Process all text files in folder and return DataFrame"""
```

Cette fonction, prend en paramètre le chemin d'un dossier `folder_path`. Elle traite tous les fichiers `.txt` contenus dans ce dossier.

```
all_data = []
```

On initialise une liste vide pour stocker les données extraites de chaque fichier.

On crée ensuite une boucle qui **parcourt** tous les fichiers `.txt` du dossier, lit leur contenu, extrait les données avec `extract_data`, puis stocke les résultats dans une liste `all_data`.

```
for filename in os.listdir(folder_path):  
    if filename.endswith('.txt'):
```

On parcourt tous les fichiers du dossier. On filtre uniquement les fichiers .txt (issus de l'OCR)

```
filepath = os.path.join(folder_path, filename)
with open(filepath, 'r', encoding='utf-8') as file:
    raw_text = file.read()
```

On construit le chemin complet du fichier, On l'ouvre et on lit son contenu dans raw_text.

Après, On utilise la fonction extract_data (déjà défini) pour extraire les données importantes (numéro de facture, montant, etc.) On ajoute le **nom du fichier source** (pour savoir d'où viennent les données). Puis on ajoute le dictionnaire extracted à la liste all_data.

```
extracted = extract_data(raw_text)
extracted['source_file'] = filename
all_data.append(extracted)
```

Une fois tous les fichiers traités, on transforme la liste de dictionnaires en un DataFrame Pandas (tableau tabulaire). Une fois tous les fichiers traités, on transforme la liste de dictionnaires en un DataFrame Pandas (tableau tabulaire).

```
df = pd.DataFrame(all_data)
```

On ajoute ces deux colonnes Et on renvoie le Dataframe final

```
df['vat'] = df['net_worth'] * 0.1
df['gross_worth'] = df['net_worth'] + df['vat']
```

Ce bloc "main execution" appelle la fonction process_folder et stock le résultat dans Df, cette partie exécute toutes les étapes précédentes.

```
# Main execution
if __name__ == "__main__":
    folder_path = 'text_ocr' # Change this if needed
    df = process_folder(folder_path)
```

On vérifie l'existence des valeurs manquantes

```
# Check for missing values
print("\nMissing Values Summary:")
print(df.isna().sum())
```

Et voici les résultats :

```

Missing Values Summary:
invoice_number      0
date                0
net_worth           0
tax_id_1            0
tax_id_2            0
iban                0
seller_city         0
seller_state        0
seller_zip          0
client_city         0
client_state        0
client_zip          0
seller_name         0
seller_address      0
client_address      0
client_name         0
source_file         0
vat                 0
gross_worth         0
dtype: int64

```

Figure 5: Résultats du test des valeurs manquantes

Observation: Le tableau ci-dessus montre que toutes les colonnes du DataFrame final sont complètes : aucune valeur n'est manquante. Cela indique que les expressions régulières utilisées pour l'extraction sont robustes et ont permis de capturer les informations attendues dans chaque fichier texte analysé

6 Structuration des résultats

6.1 Fichier Csv

Pour structurer les résultats, nous avons enregistré les données extraites dans des fichiers au format CSV, facilitant ainsi leur traitement et leur exportation. De plus, nous avons intégré un affichage sous forme de tableau via une interface, offrant ainsi une visualisation claire et intuitive des données.

extracted_invoice_data.csv												
	A	B	C	D	E	F	G	H	I	J	K	L
	invoice_number	date	net_worth	tax_id_1	tax_id_2	iban	seller_city	seller_state	seller_zip	client_city	client_state	client_zip
1	17045625	11/01/2017	28.24	903-94-7610	913-96-2620	GB43AYYU1339240	Port Jenniferfurt	ID	1419	Port Jenniferfurt	ID	1419
2	61356291	09/06/2012	192.81	949-64-9105	939-96-6477	GB50ACIE59715036	Smithmouth	NC	26872	Smithmouth	NC	26872
3	49565075	10/28/2019	87.94	964-95-3813	909-75-5482	GB79WCIS52326461	Bernardville	VA	28211	Bernardville	VA	28211
4	95611677	07/19/2016	958.27	973-97-5084	952-81-0185	GB95MPFV8855206	Reginaport	AZ	6940	Reginaport	AZ	6940
5	49622251	02/21/2015	247.94	943-88-3422	992-86-6676	GB32B5VJ1856392	East Amadahaven	CT	79669	East Amadahaven	CT	79669
6	26020078	11/19/2019	105.93	944-91-8763	942-86-9388	GB93YUAY4678420	DPO	AA	74927	DPO	AA	74927
7	34069390	05/09/2016	3884.47	909-77-8706	949-73-3279	GB49JRMV3748835	Lale Branaport	LA	6730	Lale Branaport	LA	6730
8	42465586	03/15/2012	194.92	923-88-8185	970-99-7937	GB47NOKH4542051	Port Sarah	NM	29233	Port Sarah	NM	29233
9	72794429	03/15/2021	259.98	932-82-1959	945-97-3734	GB97ZAJ81744871	Williamborough	AZ	96941	Williamborough	AZ	96941
10	94689394	01/12/2015	33776.06	943-70-0376	963-73-0110	GB67GYBL2140385	New Marilyn	WA	49740	New Marilyn	WA	49740
11	62416513	05/05/2013	377.95	999-92-4501	975-83-3913	GB02ZJHD1918685	South Jacqueline	WI	96616	South Jacqueline	WI	96616
12	46403876	02/06/2020	4198.66	949-89-2869	930-76-2809	GB86ESDN6632772	Doyreport	SC	14345	Doyreport	SC	14345
13	55043266	03/15/2012	614.66	978-96-9196	982-76-3791	GB36JQJN6956352	Huchinmandie	MT	30159	Huchinmandie	MT	30159
14	11158119	03/23/2019	119.98	988-96-5242	943-86-6756	GB83TTERF94522624	Moore LelUSN	Tha	33047	Moore LelUSN	Tha	33047
15	45142412	02/02/2014	31111.67	951-63-5457	925-97-4219	GB04DASF914734	Port Millemouth	NH	17585	Port Millemouth	NH	17585
16	65321852	04/11/2021	194.76	930-79-7945	957-82-7504	GB85KPMH5830021	New Ann	NE	54136	New Ann	NE	54136
17	53695994	10/25/2015	1829.55	907-96-2126	950-83-8341	GB23EWMU0210151	Kinsport	OK	7575	Kinsport	OK	7575
18	8268866	03/30/2012	33567.47	974-70-8786	951-73-7619	GB90GZ97906787	Markfurt	NH	79626	Markfurt	NH	79626
19	65651876	05/06/2021	50.6	963-79-1859	939-76-1432	GB10CDOB8897902	USN/ MillerPO	AE	13638	USN/ MillerPO	AE	13638
20	39280409	07/06/2014	1606.67	938-79-9168	965-83-8071	GB39KJUT706916	Deniaschire	KY	95233	Deniaschire	KY	95233
21	57262990	11/25/2016	24.66	995-67-4158	901-95-1569	GB37JTAK6451628	South Maria	AR	41635	South Maria	AR	41635
22	87278675	07/06/2019	68.74	954-89-4076	936-94-3648	GB96NEJ77423451	Joshabury	NM	79134	Joshabury	NM	79134
23	43089394	08/17/2014	47093.65	921-77-6080	971-98-5368	GB24TUNC2993959	Hellthum	HI	95143	Hellthum	HI	95143
24	15203117	05/10/2019	23.2	903-87-8854	933-80-9316	GB12YHKB885	West Erik	DE	14910	West Erik	DE	14910
25	53737767	12/08/2011	186.82	908-76-0702	910-72-9133	GB71DEIS48771	West Valericheder	AZ	99439	West Valericheder	AZ	99439
26												

Figure 6: Données extraites

6.2 Fichier Json

En plus de l'exportation des données extraites au format CSV, nous avons également mis en place une génération automatique d'un fichier au format JSON. Ce format permet une structuration hiérarchique des informations, ce qui facilite leur intégration dans des systèmes web ou des bases de données NoSQL. Le fichier JSON conserve la même logique d'organisation que le CSV, mais offre une flexibilité supplémentaire pour des traitements automatisés ou pour une visualisation dans des interfaces modernes.

Voici un aperçu de ce fichier :

```
"client_info": {
  "name": "Warren Group",
  "address": "60159 Betty Place Apt. 254",
  "city": "Port Willieville",
  "state": "VA",
  "zip": "62901",
  "tax_id": "943-78-2854"
},
"items": [
  {
    "item_no": "1",
    "description": "Super Nintendo Entertainment",
    "quantity": 5.0,
    "unit_price": 129.99,
    "net_price": 649.95,
    "vat_percentage": "10%"
  },
  {
    "item_no": "2",
    "description": "Nintendo Gameboy Color",
    "quantity": 4.0,
    "unit_price": 120.0,
    "net_price": 480.0,
    "vat_percentage": "10%"
  }
],
```

Figure 7: Aperçu du fichier JSON

6.3 Base de données

Pour organiser les données extraites des factures, nous avons conçu une base de données relationnelle, stockée sous le format SQLite dans un fichier nommé `invoice_schema_dump.sql`. Ce modèle relationnel permet de structurer les informations de manière claire et d'éviter les doublons. Il comprend trois tables principales : `Invoices`, `Clients` et `Sellers`, comme illustré ci-dessous.

Invoice Database Schema

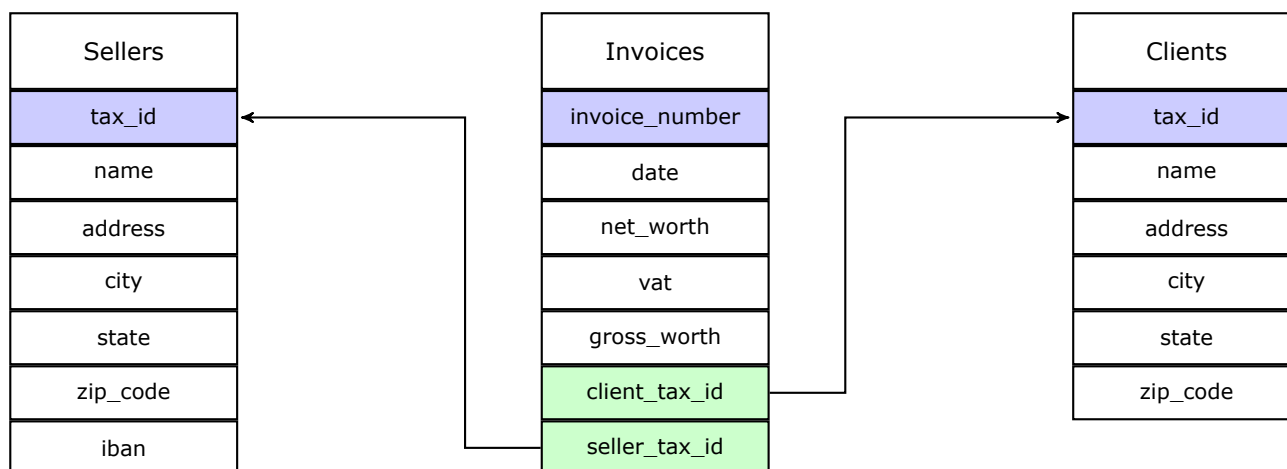


Figure 8: Schéma relationnel de la base de données des factures

La table `Invoices` stocke les informations essentielles d'une facture, telles que le numéro, les montants nets et bruts, et les identifiants des clients et vendeurs via des clés étrangères. Les tables `Clients` et `Sellers` contiennent respectivement les détails des clients et vendeurs.

6.3.1 Requêtes SQL sur la base

Pour tester et manipuler les données, nous utilisons le site sqliteonline.com, qui permet d'exécuter des requêtes SQL directement sur notre base. Voici quelques exemples de requêtes illustrant l'exploitation de la base de données.

- **1. Sélectionner toutes les factures avec un patrimoine net (`net_worth`) supérieur à 10 000 :**

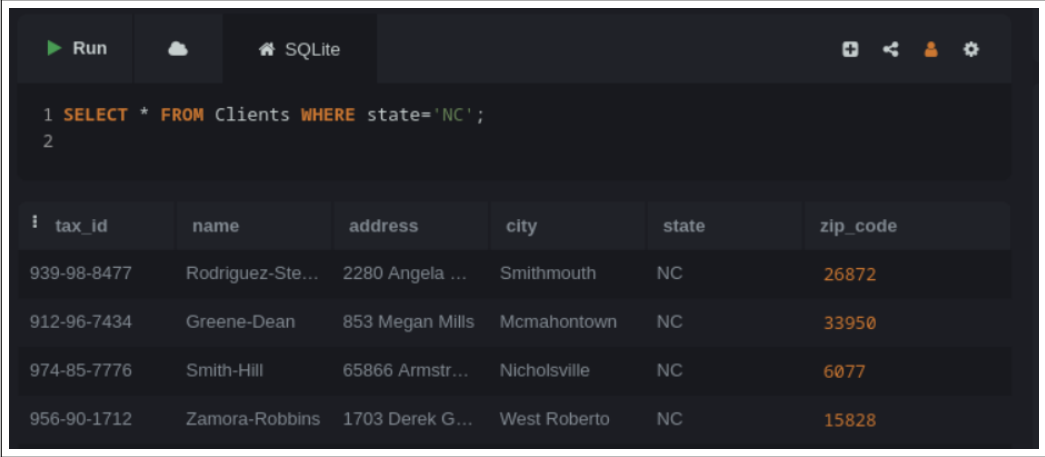


```
1 SELECT * FROM Invoices WHERE net_worth > 100000;
2
```

invoice...	date	net_worth	vat	gross_wo...	client_tax...	seller_tax_id
94133236	12/12/2019	103986.8	10398.68	114385....	937-75-3172	980-76-7923
96051364	04/28/2020	143572....	14357.2...	157929....	963-87-9620	914-85-0938
14287592	03/05/2021	135550....	13555.0...	149105....	991-91-0394	920-81-8279

Figure 9: Requête sur les factures ayant un patrimoine net élevé

- **2. Sélectionner les clients résidant dans l'état NC :**



```
1 SELECT * FROM Clients WHERE state='NC';
2
```

tax_id	name	address	city	state	zip_code
939-98-8477	Rodriguez-Ste...	2280 Angela ...	Smithmouth	NC	26872
912-96-7434	Greene-Dean	853 Megan Mills	Mcmahontown	NC	33950
974-85-7776	Smith-Hill	65866 Armstr...	Nicholsville	NC	6077
956-90-1712	Zamora-Robbins	1703 Derek G...	West Roberto	NC	15828

Figure 10: Requête sur les clients de l'état NC

7 Interface utilisateur

Pour la partie Python, nous avons utilisé les bibliothèques **streamlit** pour créer une interface utilisateur simple, et **pytesseract** pour intégrer les fonctionnalités de reconnaissance optique de caractères (OCR).

Le moteur **Tesseract** a été téléchargé depuis sa page officielle ([disponible ici](#)), où l'on peut aussi consulter une documentation complète et des instructions d'installation détaillées pour différents systèmes d'exploitation.

Sur Windows, après l'installation, le fichier `tesseract.exe` se trouve généralement dans le dossier suivant : `C:\Program Files\Tesseract-OCR`. Ce chemin a été ajouté à la variable d'environnement `PATH` pour que le système puisse le reconnaître automatiquement.

Nous avons aussi ajouté cette ligne dans le fichier `interface.py` pour spécifier explicitement le chemin de l'exécutable Tesseract :

```
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'
```

Ensuite, l'interface a été lancée à l'aide de la commande suivante dans le terminal :

```
streamlit run interface.py
```

L'interface permet d'ajouter plusieurs fichiers à traiter simultanément, soit par glisser-déposer dans la zone prévue, soit via le bouton de sélection. Cette étape précède l'extraction des données OCR.

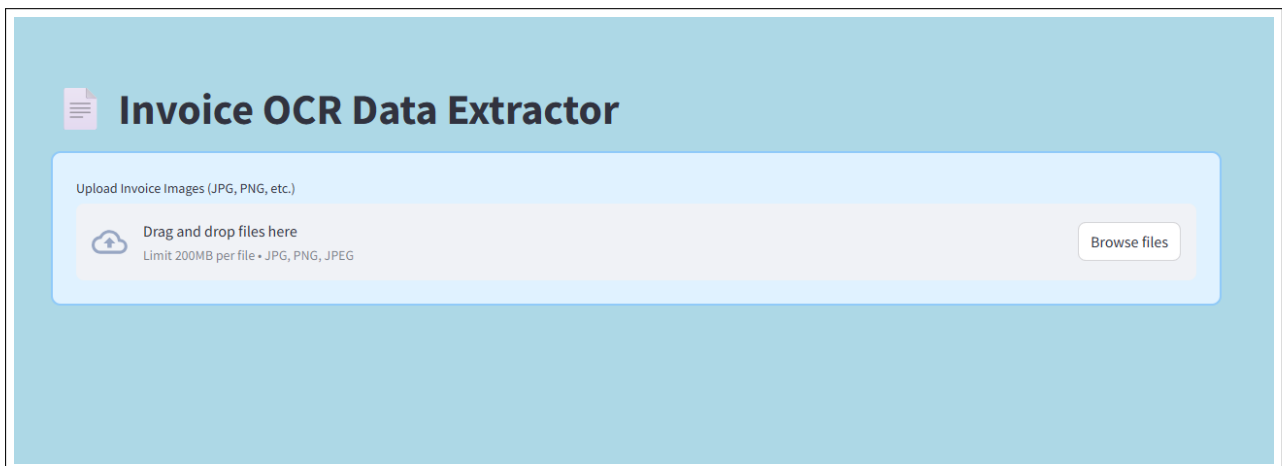


Figure 11: Ajout des fichiers à traiter

Un fichier traité affiche les données extraites sous forme de tableau. L'utilisateur peut les examiner ou les télécharger au format CSV.

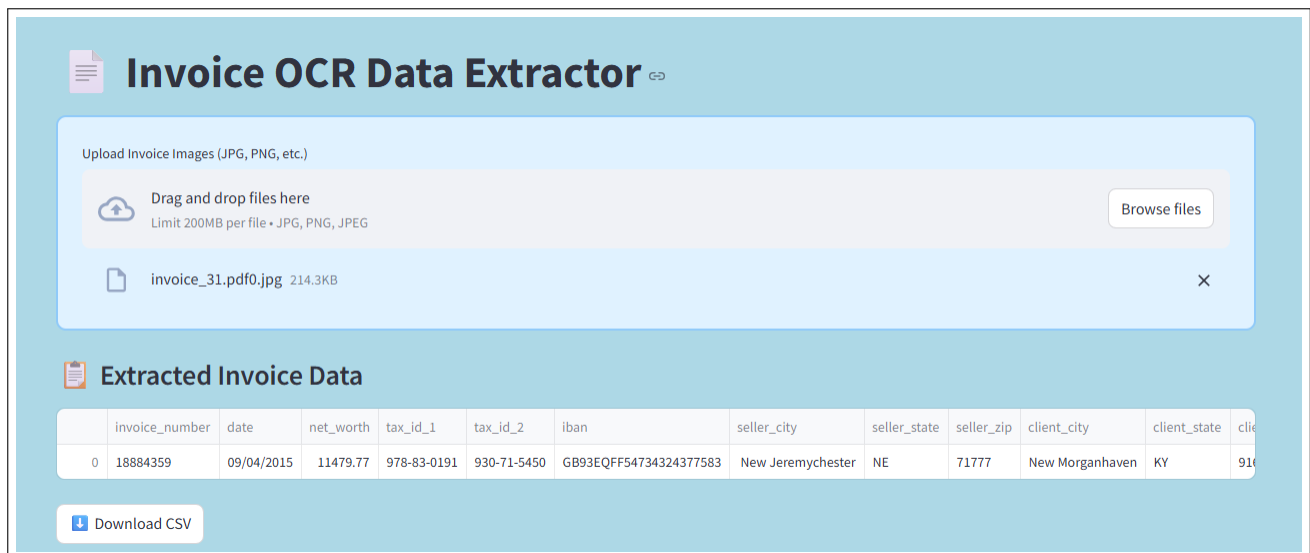


Figure 12: Résultat de l'extraction OCR

7.1 Fonctionnalités Principales

- **Recherche:** Filtrage des résultats dans le fichier CSV
- **Affichage:** Organisation optimisée des factures pour une meilleure lisibilité
- **Export:** Téléchargement des données au format CSV ou SQL base des données

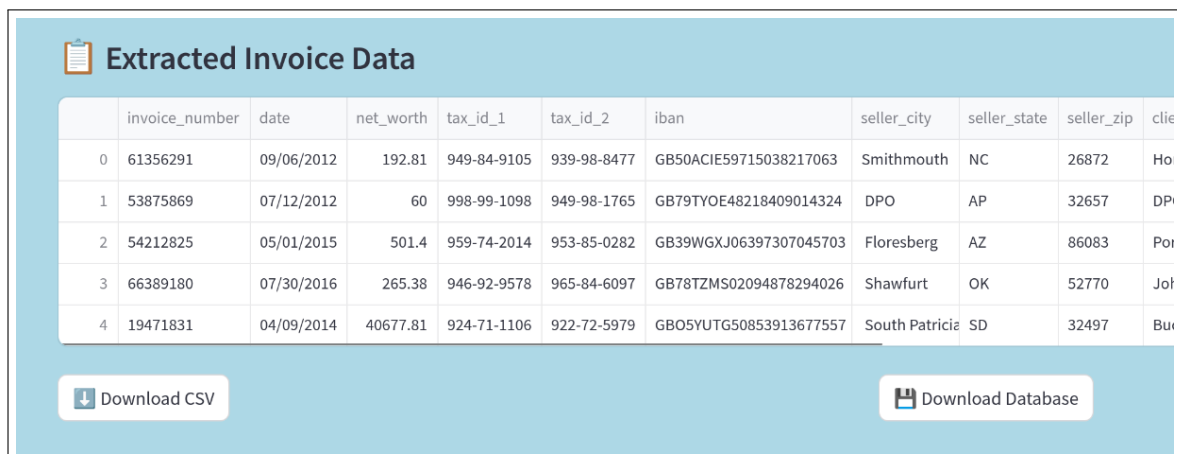


Figure 13: Exploration interactive des factures extraites

7.2 Démonstration Vidéo

Une vidéo explicative est disponible, montrant le fonctionnement complet de l'application :

- Processus d'upload des fichiers
- Extraction OCR en temps réel
- Navigation dans les résultats
- Export des données

Voici la vidéo explicative : [cliquez ici pour la visionner](#).

8 Conclusion

Ce projet nous a permis de comprendre concrètement comment automatiser l'extraction d'informations à partir de documents scannés, comme les factures. En combinant l'OCR avec les expressions régulières, nous avons appris à détecter et extraire des éléments clés tels que le numéro de facture, la date ou le montant total. Nous avons également structuré ces données dans des formats exploitables (CSV, JSON) et mis en place une interface simple pour les afficher. Ce travail nous a aidés à renforcer nos compétences en traitement de texte, en Python, et en conception d'interfaces.